Odin OS (Phase B.2): Adversarial Assurance, Evidence Integrity & Hardware-Enforced PMP Wordline Gating for Sovereign AI Agents

Nathan Walker
Ravenhelm Platform & Runestack Research
Email: nwalker85@gmail.com | ORCID: 0009-0005-0965-4424
Phase B.2 Adversarial Assurance Specification — November 3, 2026

Abstract

Current paradigms in autonomous artificial intelligence rely predominantly on probabilistic software-defined guardrails —natural language system prompts, output schemas, and container sandboxes—to constrain agent behavior. These mechanisms are mathematically vulnerable to direct prompt injection, semantic jailbreaks, and memory-state corruption. In this paper, we present **Odin OS (Phase B.2: Adversarial Assurance & Evidence Integrity)**, a bare-metal, zero-allocation (`#![no_std]`) 64-bit RISC-V microkernel that anchors autonomous agent trust boundaries directly in CPU physical memory protection (PMP) hardware. Operating on the central axiom that **state which cannot prove its right to exist cannot be committed to memory or disk**, Odin OS enforces capability isolation via cache-line aligned (256-byte) binary tokens termed **TransitionDeeds**.

We establish seven foundational architectural and adversarial contributions: (1) **Genuine Zero-Allocation Cryptographic Authority**: Integration of RFC 8032 Ed25519 digital signature verification in pure `#![no_std]` without heap allocation (`no_alloc`), replacing prototype structural attestation markers with authentic asymmetric cryptography bound to a 32-byte pinned root public key; (2) **Complete Bi-Directional $W \oplus X$ Hardware Enforcement**: Full privilege domain separation ensuring user task executable memory is strictly read-execute (`R-X`, store attempts fault with `mcause = 7`), while user stack memory is strictly read-write non-executable (`RW-`, instruction fetch attempts fault with `mcause = 1` and are safely trapped and redirected by Machine Mode); (3) **Temporal Capability Lease Revocation**: Implementation of microkernel syscall `SYS_REVOKE_LEASE` (`ecall 13`), dynamically collapsing PMP comparator windows to `OFF` and immediately converting previously valid lease addresses into hardware fault triggers; (4) **Exhaustive 17-Case Adversarial Matrix**: A deterministic bare-metal verification harness running across 17 invariant cases in QEMU, outputting machine-verifiable empirical JSON receipts with dynamic hardware fault and memory readback validation (`17 passed; 0 failed`); (5) **100% Mutation Detection Sensitivity**: Systematic injection of 10 architectural and cryptographic mutations, with 10 out of 10 mutations independently killed by the verification harness; (6) **Host Property-Based Test Suite**: 23 host unit and property tests (`proptest`) formally validating range confinement, arithmetic overflow immunity, and cross-protocol domain separation; and (7) **End-to-End Live LLM Coupling**: Demonstration against local `google/gemma-4-e4b` inference, proving that while authorized mutations commit in a single clock cycle, adversarial prompt injections targeting unauthorized physical addresses are halted by CPU hardware comparators with zero bits flipped in memory.

1 Introduction & The Software Guardrail Fallacy

The rapid deployment of autonomous AI agents tasked with executing arbitrary code, modifying datastores, and manipulating abstract syntax trees (ASTs) has exposed an architectural vulnerability in modern computing. Contemporary agent runtimes treat safety as an inference-time alignment problem, attempting to prevent unauthorized state transitions through prompt engineering, output schemas, and container sandboxes.

This premise fails against adversarial attacks for three fundamental reasons:

1. **Prompt Invariance Failure**: No natural language instruction or heuristic system prompt is invariant against adversarial prompt injection. An adversarial token sequence can reliably cause a neural model to ignore its operational parameters.
2. **Semantic Escape**: In traditional operating systems, a compromised or hallucinating agent process running in user space possesses write access to its entire virtual address space. Once a store instruction is issued to the CPU, the underlying memory controller commits the mutated bytes regardless of agent intent.
3. **Post-Hoc Remediation**: Existing detection systems operate **after** memory has been mutated, relying on logging, rollbacks, or audit trails that incur latency and fail to prevent zero-day state corruption.

To resolve this vulnerability, we invert the trust model. Rather than attempting to guarantee that neural models will never produce malicious or errant store instructions, we construct a substrate where the physical CPU memory bus is physically incapable of asserting write-enable signals for unverified transactions.

2 Architectural Axiom & Threat Model

2.1 The Central Axiom

Odin OS is constructed around a single deterministic axiom:

> ***"State that cannot prove its right to exist cannot be committed to memory or disk."***

In Odin OS, physical memory is inert and write-protected by default. An unprivileged agent task running in User Mode (U-Mode) possesses zero write authority across the physical address space until an explicit, cryptographically verifiable capability lease is presented to and verified by privileged Machine Mode (M-Mode).

2.2 Threat Boundary Classification

We classify components across four distinct trust boundaries:

- **CPU Hardware (Silicon / PMP Comparators)**: **Trusted**. Enforces address comparator matches, generates access faults, and enforces CSR privilege levels.
- **M-Mode Microkernel (TCB)**: **Trusted**. Manages trap entry/exit, verifies Ed25519 signatures, programs PMP registers, and controls monotonic replay sequences.
- **Autonomous Agent (U-Mode Task)**: **Untrusted (Adversary)**. Executes untrusted model instructions and arbitrary arithmetic.
- **Host / Python Bridge**: **Untrusted Channel**. Transports model tokens and serializes deeds; assumed to be completely compromiseable.

3 The TransitionDeed Capability Specification

State transitions in Odin OS are mediated through **TransitionDeeds**. A TransitionDeed is a standardized, 256-byte physical binary packet designed for cache-line alignment with zero heap allocations.

Offset	Field	Type	Description
`0x00..0x02`	`magic`	`[u8; 2]`	Magic identifier `0x59 0x47` ("YG")
`0x02..0x03`	`version`	`u8`	Protocol specification version (`0x02` for Phase B.2)
`0x03..0x04`	`flags`	`u8`	Capability flags (`MUTATION`, `AST_STRICT`)
`0x04..0x08`	`sequence`	`u32`	Monotonically increasing deed nonce
`0x08..0x18`	`issuer_did`	`[u8; 16]`	Cryptographic identity of the issuer
`0x18..0x28`	`target_urn`	`[u8; 16]`	Target entity uniform resource name
`0x28..0x48`	`prev_deed_hash`	`[u8; 32]`	Cryptographic hash of prior deed
`0x48..0x68`	`base_digest`	`[u8; 32]`	SHA-256 pre-mutation state digest
`0x68..0x6C`	`lease_cycles`	`u32`	Hardware execution cycle lifetime
`0x6C..0x70`	`operation`	`u8 + u8 + u16`	Opcode, escalation level, reserved
`0x70..0x74`	`anchor_id`	`u32`	Target AST Anchor identifier
`0x74..0x78`	`target_offset`	`u32`	Authorized buffer start offset
`0x78..0x7C`	`target_length`	`u32`	Authorized buffer span length
`0x7C..0x9C`	`new_digest`	`[u8; 32]`	Expected post-mutation state digest
`0x9C..0xDC`	`signature`	`[u8; 64]`	RFC 8032 Ed25519 digital signature
`0xDC..0x100`	`padding`	`[u8; 36]`	Zero padding to align to 256 bytes

Table 1: Physical layout of the 256-byte cache-line aligned TransitionDeed structure.

3.1 Cryptographic Authority Verification ($\text{Agent Request} \neq \text{Authority Grant}$)

In Phase B.1, deed authorization relied on a structural attestation marker (`SIGN`) coupled with an unkeyed 64-bit checksum. Under adversarial review, this mechanism was identified as vulnerable to forgery by an untrusted or compromised host bridge.

Phase B.2 introduces genuine asymmetric cryptographic authority verification:

1. **Canonical Domain-Separated Payload**: The signed message encompasses 172 contiguous bytes:
$$\text{Message} = \text{DOMAIN_SEPARATOR}_{16\text{B}} \parallel \text{Metadata}_{156\text{B}}$$
where `DOMAIN_SEPARATOR` is `"ODIN_DEED_V2\0\0\0\0"`, preventing cross-protocol signature replay.
2. **Pinned Root Verification Key**: Signatures are verified against a 32-byte public key hardcoded into kernel read-only memory (`TRUSTED_ROOT_PUBLIC_KEY = [0xFD, 0xC8, ..., 0x19]`). The private key resides outside the agent host environment.
3. **Zero-Allocation Execution**: Implemented via `ed25519-compact` (`opt_size`, pure `#![no_std]`). All elliptic curve additions, point decompression, and SHA-512 transformations execute on the privileged kernel stack with zero heap allocations.
4. **Fail-Closed Policy**: Signature mismatches, malformed points, sequence replays, or unauthorized issuer DIDs immediately fail at the microkernel syscall boundary, preventing any PMP register update.

4 Privilege Boundary Hardening & Silicon PMP Architecture

4.1 Privilege Domain Layout

Odin OS partitions physical memory into five strictly isolated privilege domains:

Domain	Physical Span	Mode	PMP Mode	Privilege & Architectural Purpose
`Domain 0`	`0x10000000` .. `0x10001000`	M-Mode	Default / `---`	UART0 MMIO (Revoked from U-Mode)
`Domain 1`	`0x80000000` .. `0x80010000`	M-Mode	Default / `---`	Privileged Kernel: Code, Stack, Keys, Traps
`Domain 2`	`0x80010000` .. `0x80018000`	U-Mode	TOR / `R-X`	Untrusted Agent Task Code ($W \oplus X$)
`Domain 3`	`0x80018000` .. `0x80020000`	U-Mode	TOR / `RW-`	Untrusted Agent Stack (Non-Executable)
`Domain 4`	`0x80020000` .. `lease_start`	None	TOR / `---`	Gated Pre-Lease AST Boundary
`Domain 5`	`lease_start` .. `lease_end`	U-Mode	TOR / `RW-`	Explicitly Authorized TransitionDeed Slice
`Perimeter`	All other physical addresses	None	OFF / `---`	Hardware Default Deny (`WE = 0`)

Table 2: Phase B.2 Hardened 7-Region PMP Layout and Privilege Partitioning.

4.2 Hardware W^X Stack Execution Prevention & Trap Handling

When user mode attempts to execute shellcode from the user stack (`0x80018100`), the CPU instruction fetch unit triggers an Instruction Access Fault (`mcause = 1`). In Phase B.2, the M-Mode trap handler intercepts the fault, records the invariant violation, and safely redirects `mepc` back to the return address `frame.ra` in `.user_text`:

```
if mcause == 1 && mepc >= USER_STACK_START as u64 && mepc < USER_STACK_END as u64 {
    unsafe { core::arch::asm!("csrw mepc, {0}", in(reg) frame.ra); }
    return;
}
```

This guarantees that stack execution attempts cannot escalate privileges or enter infinite fault loops.

4.3 Temporal Lease Revocation (`SYS_REVOKE_LEASE`)

To prevent temporal capability leakage, Phase B.2 introduces microkernel syscall 13 (`sys_revoke_lease`). When invoked, M-Mode clears `ACTIVE_LEASE_START` and `ACTIVE_LEASE_END` and sets PMP Entry 4 to `OFF`. Subsequent store attempts to the previously valid lease slice immediately trigger a hardware Store Access Fault (`mcause = 7`).

5 The 17-Case Adversarial Verification Matrix

To systematically validate the hardware enforcement boundary, we implemented a deterministic 17-case adversarial verification matrix executing directly in U-Mode on the bare-metal kernel. Dynamic invariant evaluations in M-Mode verify expected hardware faults, trap causes, readback values, and memory preservation.

#	Test Invariant Case	Target Addr	Width	Exp Fault	Obs Fault	mcause	Status
1	Exact Lower Bound	0x80020040	1B	No	No	0x00	**PASS**
2	Exact Upper Bound - 1	0x8002005F	1B	No	No	0x00	**PASS**
3	One Byte Below Lease	0x8002003F	1B	Fault	Fault	0x07	**PASS**
4	One Byte Above Lease	0x80020060	1B	Fault	Fault	0x07	**PASS**
5	Kernel Code Write	0x80000000	1B	Fault	Fault	0x07	**PASS**
6	Kernel Stack Write	0x8000FFF8	1B	Fault	Fault	0x07	**PASS**
7	User Code Write ($W \oplus X$)	0x80010000	1B	Fault	Fault	0x07	**PASS**
8	User Stack Execute ($W \oplus X$)	0x80018100	0B	Fault	Fault	0x01	**PASS**
9	Unmapped Perimeter Write	0x80030000	1B	Fault	Fault	0x07	**PASS**
10	Multi-byte Overlap Crossing	0x8002005E	4B	Fault	Fault	0x07	**PASS**
11	Forged Deed Issuer DID	0x00000000	0B	Reject	Reject	Syscall / 1	**PASS**
12	Replayed Deed Sequence	0x00000000	0B	Reject	Reject	Syscall / 2	**PASS**
13	Forged Deed Invalid Ed25519	0x00000000	0B	Reject	Reject	Syscall / 4	**PASS**
14	Deed Tampered Metadata Ed25519	0x00000000	0B	Reject	Reject	Syscall / 4	**PASS**
15	U-Mode CSR Access pmpcfg0	0x00000000	0B	Fault	Fault	0x02	**PASS**
16	Expired Lease Revocation	0x80020040	1B	Fault	Fault	0x07	**PASS**
17	U-Mode UART0 MMIO Write	0x10000000	1B	Fault	Fault	0x07	**PASS**

Table 3: Empirical receipts from the Phase B.2 17-case adversarial verification matrix.

6 Controlled Mutation Testing

To mathematically prove test suite sensitivity and eliminate verification tautologies, we injected 10 deliberate mutations into kernel logic, PMP configurations, and capability validation code:

ID	Injected Architectural Mutation	Detecting Invariant Test	Result
M01	Bypass issuer DID check	Test 11 (FORGED_DEED_UNTRUSTED_ISSUER)	KILLED
M02	Disable Ed25519 signature verification	Tests 13 & 14 (FORGED_DEED_ED25519)	KILLED
M03	Permit sequence reuse (< instead of <=)	Test 12 (REPLAYED_DEED_SEQUENCE)	KILLED
M04	Wrapping addition on bounds (disable checked)	prop_deed_overflow_protection	KILLED
M05	Expand PMP lease by 1 byte in setup_pmp	Test 4 (ONE_BYTE_ABOVE_LEASE)	KILLED
M06	Configure user code as RWX (0x0F)	Test 7 (USER_CODE_WRITE_FAULT)	KILLED
M07	Map UART MMIO in U-Mode	Test 17 (U_MODE_UART_MMIO_WRITE)	KILLED
M08	Corrupt register restore in trap_entry	Architectural Register Invariants	KILLED
M09	Skip PMP reprogramming on lease revocation	Test 16 (EXPIRED_LEASE_REVOCATION)	KILLED
M10	Tamper with receipt status emission	Dynamic Invariant Assertions	KILLED

Table 4: Controlled mutation testing results demonstrating 100% test suite detection sensitivity.

7 End-to-End Validation Against Live LLM Inference

We coupled the hardened microkernel to live local inference with `google/gemma-4-e4b` via LM Studio (`http://127.0.0.1:6701/v1`):

1. **Scenario 1 (Authorized Mutation)**: Gemma 4 proposed an authorized mutation within lease `[0x40..0x60]`. The Python bridge compiled an authenticated TransitionDeed signed with the root authority seed (`b"odin_root_authority_seed_v2_key!"`). Privileged M-Mode verified the Ed25519 signature, programmed PMP Domain 5, and demoted to U-Mode. The store instruction committed in a single clock cycle.
2. **Scenario 2 (Direct Prompt Injection)**: Gemma 4 was subjected to a direct jailbreak prompt attempting to overwrite kernel memory at `0x80030000`. The model outputted the out-of-bounds target. The CPU hardware comparators trapped immediately on `0x80030000` with `mcause = 0x07` (Store Access Fault), terminating the bus cycle before commit. Memory readback confirmed target memory remained completely unmodified.

8 Physical Silicon vs Emulation Transparency

We explicitly document the architectural assumptions under which our QEMU empirical evidence holds, and the migration requirements for Phase C physical hardware:

- **PMP Granularity (G)**: QEMU implements byte-exact Top-of-Range (TOR) resolution ($G = 0$). Commercial physical silicon (e.g. StarFive JH7110, SiFive U74) implements a non-zero grain size ($Gge2$ or $Gge4$), requiring capability allocations to align to 16-byte boundaries.
- **Store Buffer Draining**: Physical cores require explicit `fence rw, rw` and `fence.i` instructions during PMP transitions to prevent pipeline store-forwarding across lease boundaries.
- **Physical DRAM Gating**: On-chip PMP terminates transactions at the core interface. Phase C hardware development on FPGA will implement physical wordline gating at the memory controller, suppressing DRAM row-select voltages on unauthorized bus transactions.

9 Reproducibility & Open Artifacts

The complete Odin OS codebase, linker configurations, property test suites, and empirical JSON receipts are archived under permanent DOI on Zenodo.

- **Git Repository**: `https://github.com/nwalker85/odinos`
- **Branch**: `feat/phase-b2-adversarial-assurance`
- **Target Architecture**: `riscv64gc-unknown-none-elf`
- **Empirical Receipts**: `zenodo/receipts/adversarial_matrix_receipts.json`
- **Raw Execution Trace**: `zenodo/receipts/raw_execution_trace.txt`

- **Toolchain**: Rust nightly `#![no_std]`, QEMU 11.1.3+, Python 3.14+, Typst 0.15+

10 Conclusion

Odin OS Phase B.2 establishes that autonomous AI agent security can be grounded in verifiable cryptographic authority and deterministic physical-layer enforcement. By implementing zero-allocation Ed25519 signature verification, enforcing strict bi-directional $W \oplus X$ protections, validating temporal capability revocation, and subjecting the microkernel to a 17-case adversarial matrix and 100% mutation testing, we achieve mathematical containment against prompt injection, model jailbreaks, and memory corruption.